SOLID prinsiplərinin səbəbi və tətbiqi

SOLID obyekt yönümlü dizaynda beş prinsipin baş hərfləridir. Məqsəd proqramın dəyişiklik zamanı hansı hissələrinin təsirlənəcəyini nəzarətdə saxlamaqdır. Prinsiplər sinif sayını artırmaq üçün məcburi qaydalar deyil. Düzgün tətbiq konkret dəyişiklik səbəbini, müqaviləni və asılılığın istiqamətini aydınlaşdırır.

İlkin `files/03_di_architecture.pdf` materialında S, O və D nümunələri var idi, L və I isə qısa verilirdi. Bu dərs bütün beşini ayrıca problem, düzgün model və sərhədlərlə tamamlayır. “Override exception atırsa həmişə LSP pozulub” hökmü də dəqiqləşdirilir.

Mündəricat

SOLID prinsiplərinin səbəbi və tətbiqi **1**

1 Dizaynı qiymətləndirmək üçün sözlük 2

2 S Single Responsibility Principle 2

3 O Open Closed Principle 2

4 L Liskov Substitution Principle 3

5 I Interface Segregation Principle 4

6 D Dependency Inversion Principle 4

7 Hamısını bir sifariş axınında birləşdirək 5

8 Müsahibə sualları və məşq 5

9 Mənbələr 6

1 Dizaynı qiymətləndirmək üçün sözlük

Module əlaqəli kod vahididir; sinif, kitabxana və ya paket ola bilər. **Cohesion** bir vahidin içindəki hissələrin eyni məqsədə xidmət etmə dərəcəsidir. **Coupling** vahidlər arasındakı asılılıq dərəcəsidir. Yüksək uyğunluq və idarə olunan bağlılıq dəyişiklik sahəsini anlamağı asanlaşdırır.

Contract, müqavilə, metodun qəbul etdiyi giriş, qaytardığı nəticə və gözlənilən davranışdır. **Client** burada interface-i istifadə edən koddur, mütləq son istifadəçi deyil. **Refactoring** xaricdən müşahidə olunan davranışı qoruyaraq kodun daxili quruluşunu dəyişməkdir. **Abstraction** vacib davranışı göstərib texniki təfərrüatı gizlədən modeldir. Interface onun bir formasıdır.

Sinif məlumat və davranış üçün tipdir. Interface həmin tipin təqdim etdiyi əməliyyat müqaviləsidir. Inheritance, miras alma, bir tipin digər tipdən davranış götürməsidir. Composition, tərkibləmə, bir obyektin başqalarından istifadə etməsidir. Hər “is a”, yəni növ münasibəti, təhlükəsiz davranış əvəzlənməsi demək deyil.

2 S Single Responsibility Principle

SRP, Single Responsibility Principle, vahidin bir əlaqəli dəyişiklik səbəbinə xidmət etməsini istəyir. “Sinifdə yalnız bir metod olmalıdır” demək deyil. Əsas sual budur: bu kodu hansı qaydanın və ya kimin tələbinin dəyişməsi dəyişdirir?

Məsələn, `InvoiceService` hesabın vergi məbləğini hesablayır, SQL ilə saxlayır, PDF düzəldir və email göndərir. Vergi qaydası maliyyə qrupuna, saxlanma texnologiyası infrastruktur qərarına, PDF görünüşü dizayn tələbinə bağlıdır. Bunların hamısı eyni sinfi dəyişdirirsə əlaqəsiz dəyişiklik səbəbləri qarışır.

Mümkün bölgü: `InvoiceCalculator` hesabı hesablayır, `InvoiceStore` saxlayır, `InvoiceRenderer` görünüş verir, `InvoiceSender` göndərir. Bir application service bu işləri istifadə ssenarisinə görə ardıcıllaşdıra bilər. Application service istifadəçinin işi üçün mərhələləri əlaqələndirən xidmət deməkdir. Əlaqələndirmə özü də ayrıca məsuliyyət ola bilər.

Sərhəd: 3 sətirlik sadə funksiyanı 7 sinfə bölmək həmişə yaxşılaşma deyil. Vergi hesablayan sinfin bir neçə köməkçi metodu olması SRP pozuntusu sayılmır. Eyni qaydaya görə birlikdə dəyişən davranışları bir yerdə saxlamaq məntiqlidir. Robert C. Martin-in izahında əlaqəli dəyişiklik səbəbləri və aktorlar vacibdir; aktor kodun davranışına tələb qoyan insan və ya biznes roludur.

3 O Open Closed Principle

OCP, Open Closed Principle, müəyyən dəyişiklik istiqamətində sistemi yeni davranışla genişləndirə bilməyi, sabit əsas məntiqə isə az toxunmağı hədəfləyir. Bütün gələcək dəyişikliklərə qarşı qapalı olmaq mümkün deyil. Hansı dəyişikliklərin tez-tez olacağını seçmək lazımdır.

C#

```
public interface IDiscount
{
    decimal Calculate(decimal total);
}

public sealed class VipDiscount : IDiscount
{
    public decimal Calculate(decimal total) => total * 0.20m;
}

public sealed class Checkout(IDiscount discount)
{
    public decimal Payable(decimal total)
    {
        return total - discount.Calculate(total);
    }
}
```

`decimal` pul hesablaması üçün çox istifadə olunan dəqiq onluq tipdir. `0.20m` həmin tipdə sabitdir. `Checkout` endirim strategiyasını istifadə edir, konkret faiz bilmir. Yeni `StudentDiscount` əlavə edəndə Checkout-u dəyişmədən eyni müqavilə ilə işləyə bilərsən. Composition root-da yeni strategiyanın seçilməsi və qeydiyyatın dəyişməsi normaldır; OCP “heç bir fayl heç vaxt dəyişməsin” demir.

Strategy dəyişən alqoritmi ayrıca müqavilə ilə təqdim edən pattern-dir. Hər switch pis deyil. İki sabit hal üçün sadə switch daha aydın ola bilər. Dəyişiklik tezliyi və variantların müstəqilliyi artanda strategiyaya çıxarmaq faydalıdır. Overengineering hələ olmayan ehtiyac üçün artıq mürəkkəblik yaratmaqdır.

4 L Liskov Substitution Principle

LSP, Liskov Substitution Principle, alt tipin baza tipin yerində istifadə olunanda baza müqaviləsinə güvənən kodu pozmamasını istəyir. Burada tək metod adlarının eyni olması yetmir; davranış qorunmalıdır.

Precondition metod çağırılmazdan əvvəl təmin olunmalı şərtdir. **Postcondition** metod uğurla bitəndə doğru olmalı nəticədir. **Invariant** obyektin etibarlı vəziyyətində həmişə saxladığı qaydadır. Alt tip qəbul edilən girişləri daraltmamalı, vəd edilən nəticəni zəiflətməməli və invariantları pozmamalıdır. Bu, tədris üçün sadələşdirilmiş davranış müqaviləsi çərçivəsidir.

Məsələn, `IFileStore.Write` etibarlı fayl yoluna yazmağı vəd edir. `ReadOnlyStore` həmin müqaviləni qəbul edib həmişə `NotSupportedException` atırsa çağıran kodun gözləntisini pozur. `NotSupportedException` əməliyyat dəstəklənmir xətasıdır. Daha düzgün model `IFileReader` və `IFileWriter`-ı ayırmaqdır; yalnız oxuyan obyekt writer müqaviləsini götürmür.

Ancaq exception atılması təkbaşına LSP pozuntusu deyil. Baza müqaviləsi “disk doludursa yazma xətası qaytara bilər” deyirsə alt tipin bu halda xəta atması gözlənilən davranışdır. Exception proqramın normal axınından çıxan xəta siqnalıdır. Əsas sual yeni alt tipin əvvəl qəbul olunan etibarlı vəziyyət üçün yeni, gözlənilməyən məhdudiyyət gətirib gətirməməsidir.

Klassik dəyişə bilən düzbucaqlı kvadrat nümunəsində baza tipin eni və hündürlüyü müstəqil dəyişə bilməsi müqavilədirsə, kvadratın birini dəyişəndə o birini də dəyişməsi bu gözləntini pozur. Riyazi mənada kvadrat düzbucaqlıdır, amma həmin mutable API müqaviləsi üzrə təhlükəsiz alt tip olmaya bilər. **Mutable** dəyişdirilə bilən deməkdir. Problemi immutable, yaradıldıqdan sonra dəyərləri dəyişməyən model və ya ayrıca fiqur müqaviləsi ilə həll etmək olar.

Müsahibə yoxlaması: yeni implementasiyanı götür, ona bütün köhnə müqavilə testlərini tətbiq et. Məqsəd yalnız “xəta atmır” deyil, giriş, nəticə və vəziyyət qaydalarını eyni saxlamaqdır.

5 I Interface Segregation Principle

ISP, Interface Segregation Principle, client-in istifadə etmədiyi əməliyyatlara məcburi bağlı olmamasını istəyir. Printer, skaner və faks metodları olan böyük `IMachine` sadə printeri lazımsız metodları həyata keçirməyə məcbur edə bilər.

C#

```
public interface IPrinter
{
    void Print(string text);
}

public interface IScanner
{
    byte[] Scan();
}

public sealed class SimplePrinter : IPrinter
{
    public void Print(string text)
    {
        Console.WriteLine(text);
    }
}
```

`byte[]` bayt massivi, yəni ikili məlumat ardıcıllığıdır. `SimplePrinter` scan etməyi vəd etmir. Çoxfunksiyalı cihaz hər iki interface-i həyata keçirə bilər. Bu bölgü real cihazın bacarıqları ilə müqaviləni uyğunlaşdırır.

ISP-ni “hər interface-də bir metod olmalıdır” kimi əzbərləmə. Birlikdə istifadə olunan 4 oxuma əməliyyatını `IOrderReader`-da toplamaq normal ola bilər. Interface sərhədini client-in ehtiyacı və dəyişiklik əlaqəsi müəyyən edir. Lazımsız bölgü də çağırış kodunu çətinləşdirə bilər.

ISP ilə LSP əlaqəsi: oxuya bilməyən writer və ya yaza bilməyən reader üçün nəhəng müqavilə yaratmaq LSP pozuntusuna gətirə bilər. Müqavilələri ehtiyaca görə ayırmaq təhlükəsiz əvəzlənməni asanlaşdırır.

6 D Dependency Inversion Principle

DIP, Dependency Inversion Principle, iş qaydasının SQL, email kitabxanası və HTTP detalları kimi dəyişən mexanizmlərə birbaşa bağlanmasını azaltmaq istəyir. Yüksək səviyyəli siyasət və aşağı səviyyəli detal uyğun abstraksiyaya bağlı olmalıdır. Abstraksiyanın dili isə işin ehtiyacını ifadə etməlidir.

C#

```
public interface IReceiptSender
{
    Task SendAsync(string address, CancellationToken ct);
}

public sealed class CompleteOrder(IReceiptSender sender)
{
    public Task NotifyAsync(string address,
        CancellationToken ct)
    {
        return sender.SendAsync(address, ct);
    }
}
```

Bu fraqmentdə iş xidməti SMTP bağlantısının necə açıldığını bilmir. SMTP email ötürmə protokoludur. `Task` tamamlanmanı təmsil edir, `CancellationToken` ləğv istəyini ötürür. Texniki email adapteri həmin müqaviləni yerinə yetirə bilər. Interface-in özü konkret kitabxananın bütün tiplərini tələb edirsə detal yenə daxilə sızmış ola bilər.

DI bu asılılığı constructor-la vermə üsuludur. DIP dizaynın istiqamətidir. Container-dən istifadə edən, amma konkret SQL sinfinə möhkəm bağlı xidmət DI edir, lakin DIP məqsədinə zəif xidmət edə bilər. Digər tərəfdən container olmadan manual construction ilə DIP-yə uyğun sistem qurmaq mümkündür.

7 Hamısını bir sifariş axınında birləşdirək

`Checkout` sifarişi tamamlasın. Endirim hesablamasını `IDiscount` versin, saxlanmanı `IOrderStore`, bildirişi `IReceiptSender` həyata keçirsin. SRP müxtəlif dəyişiklik səbəblərini ayırır. OCP yeni endirim variantını əlavə etməyə imkan verir. LSP hər endirim implementasiyasının müqaviləni saxlamasını tələb edir. ISP yalnız oxuyan hesabat kodunu yazma metodlarından asılı etmir. DIP iş qaydasını konkret baza və email mexanizmindən ayırır.

Bu bölgü transaction problemini avtomatik həll etmir. Sifariş bazada saxlanıb email göndərilməsi uğursuz ola bilər. Dizayn prinsipi ilə məlumatın çatdırılma zəmanəti ayrı məsələlərdir. Outbox kimi etibarlı hadisə göndərmə yanaşması lazım ola bilər; o, dəyişikliklə hadisəni eyni bazada saxlayıb sonradan göndərir.

8 Müsahibə sualları və məşq

SRP-ni necə tapırsan? Sinfin nə qədər uzun olduğundan əvvəl onun hansı bir-birindən müstəqil tələblərə görə dəyişdiyini soruşuram.

OCP xətanı düzəltməyə mane olurmu? Xeyr. Səhvi düzəltmək üçün mövcud kod dəyişir. Prinsip seçilmiş genişlənmə istiqamətində sabit məntiqə lazımsız toxunmanı azaltmaq üçündür.

Interface varsa DIP təmin olunubmu? Təkbaşına yox. Interface yanlış qatın texniki tipləri ilə doludursa və iş qaydası yenə həmin detala bağlıdırsa asılılıq istiqaməti düzəlməyib.

Məşq. Həm diskə yazan, həm email göndərən, həm endirim hesablayan sinif götür. Dəyişiklik səbəblərini yaz. Sonra endirimi ayır və iki implementasiya qur. Hər ikisinin eyni etibarlı girişlər üçün uyğun nəticə verdiyini yoxla. Sonda qeydiyyat xaricində sifariş axınının dəyişmədən yeni endirim aldığını göstər. Məqsəd sinif sayını deyil, dəyişiklik sahəsini ölçməkdir.

9 Mənbələr

- Robert C. Martin - Single Responsibility Principle
- Liskov və Wing - A Behavioral Notion of Subtyping
- Microsoft Learn - Architectural principles

Yoxlama tarixi 28 oktyabr 2026. Nümunələr müqaviləni göstərən C# fraqmentləridir. Tam biznes və texniki implementasiya ayrıca qurulmalıdır.